

SKILL EXPERIMENT 17

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of Active Job Listing and Foreground Job Switching

Aim

To implement active job listing by displaying status information, retrieving job IDs, formatting job output, updating job state information, and testing job listings.

To design foreground switching by identifying the target job, transferring terminal control, resuming processes, waiting for completion, updating job state, and testing foreground switching.

Theory

Job listing allows a shell to maintain and display information about currently active jobs. Each job is identified using a unique job ID and its status can be displayed in a formatted manner. The job state is updated whenever the process changes its execution condition.

Foreground switching allows a selected background job to receive control of the terminal. The shell identifies the target job using its job ID, transfers terminal control to the corresponding process group, and resumes the process when required. The shell waits for the foreground job to complete or change its state and then updates the job information accordingly.

Algorithm

Part A – List Active Jobs

1. Start the program.
2. Initialize the job table.
3. Store the active jobs with their job IDs and process information.
4. Retrieve the job IDs from the job table.
5. Check the current state of each job.
6. Update the state information of the jobs.
7. Format the job information for display.
8. Display the job ID, process information, and current status.

9. Test the job listing with active and completed jobs.
10. Remove or update jobs whose state has changed.
11. Display the updated list of active jobs.

Part B – Foreground Switching

1. Start the foreground switching operation.
2. Identify the target job using its job ID.
3. Locate the corresponding process or process group.
4. Transfer terminal control to the target process group.
5. Resume the selected process if it is stopped.
6. Wait for the foreground process to complete or change its state.
7. Retrieve the updated process status.
8. Update the job state in the job table.
9. Restore terminal control to the shell.
10. Test foreground switching with an active job.
11. Display the updated job state.
12. Stop the program.

CODE

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
typedef struct {
```

```
    int job_id;
```

```
    pid_t pid;
```

```
    char state[20];
```

```
} Job;
```

```
int main() {
```

```
    Job job;
```

```
    pid_t pid;
```

```
printf("Starting a background job...\n");
```

```
pid = fork();
```

```
if (pid == -1) {  
    perror("fork");  
    return 1;  
}
```

```
if (pid == 0) {  
    printf("Job ID: 1 | PID: %d | State: Running\n", getpid());  
  
    sleep(3);  
  
    printf("Job ID: 1 | Process completed.\n");  
    return 0;  
}
```

```
job.job_id = 1;
```

```
job.pid = pid;
```

```
sprintf(job.state, "Running");
```

```
printf("\nActive Job List:\n");
```

```
printf("Job ID: %d | PID: %d | State: %s\n",
```

```
        job.job_id, job.pid, job.state);

printf("\nIdentifying target job...\n");

printf("Target Job ID: %d\n", job.job_id);


printf("Transferring job to foreground...\n");

printf("Waiting for Job %d to complete...\n", job.job_id);


waitpid(pid, NULL, 0);


sprintf(job.state, "Completed");


printf("\nUpdated Job Status:\n");

printf("Job ID: %d | PID: %d | State: %s\n",

        job.job_id, job.pid, job.state);


printf("\nForeground switching completed successfully.\n");

return 0;
}
```

COMPILATION

```
hp@DESKTOP-35T9S8P:~$ nano exp2.c
hp@DESKTOP-35T9S8P:~$ gcc exp2.c -o exp2
hp@DESKTOP-35T9S8P:~$ ./exp2
Starting a background job...

Active Job List:
Job ID: 1 | PID: 524 | State: Running

Identifying target job...
Target Job ID: 1
Transferring job to foreground...
Waiting for Job 1 to complete...
Job ID: 1 | PID: 524 | State: Running
Job ID: 1 | Process completed.

Updated Job Status:
Job ID: 1 | PID: 524 | State: Completed

Foreground switching completed successfully.
hp@DESKTOP-35T9S8P:~$ |
```

Result

The active job listing and foreground switching operations were implemented successfully. Active jobs were identified using job IDs, their status information was displayed and updated, and foreground jobs were selected, given terminal control, resumed, and monitored until their state changed.